Lab 1

Run this Lab Example:

Step 1 — Select & check the dataset

Palmer Penguins: body measurements for 344 penguins of three species (Adelie, Chinstrap, Gentoo), collected on islands near Palmer Station, Antarctica.

Check	Result
Tabular, alpha-numeric	plain CSV, numbers and short text categories
Feature count	7 columns
Enough records for a first look?	Yes — 344 rows

Step 2 — Document it

Field	Answer
Dataset name	Palmer Penguins (penguins.csv, the curated/cleaned version)
Source	https://github.com/allisonhorst/palmerpenguins
Direct file	https://raw.githubusercontent.com/allisonhorst/palmerpenguins/main/inst/extdata/penguins.csv
Access date	(fill in today's date)
Licence	CC0 ("No Rights Reserved") — public domain, per the Palmer Station LTER Data Policy
Original collection	Field measurements taken by researchers by hand, on live penguins, at three islands in the Palmer Archipelago, 2007-2009

How the data was actually collected, in plain terms: Researchers with the Palmer Station Antarctica research program physically caught individual penguins, weighed them, and measured their flippers and bills (the culmen — the top ridge of the beak) with callipers, then released them. Each row is one penguin, measured once. Missing values happen when a particular measurement wasn't taken for that bird — for instance, if it wasn't safely possible to determine sex in the field that day.

Step 3 — Load it

```
import pandas as pd
import numpy as np

df =
pd.read_csv('https://raw.githubusercontent.com/allisonhorst/palmerpenguins/main/inst/extdata/penguins.csv')
df.head()
```

Step 4 — Describe its structure

```
print("Records:", df.shape[0])   # shape[0] = number of rows
```

```
print("Features:", df.shape[1])  # shape[1] = number of columns
df.dtypes                         # data type of every column (int, float, object, etc.)
```

Step 5 — Inspect data quality

```
print("--- Missing values ---")
print(df.isna().sum())                    # blanks per column

print("--- Duplicate rows ---")
print(df.duplicated().sum())              # exact copies of a row

print("--- Unique species ---")
print(df["species"].unique())             # distinct species spellings

print("--- Island counts ---")
print(df["island"].value_counts())        # rows per island

print("--- Summary statistics: measurements ---")
continuous_cols = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]
print(df[continuous_cols].describe())     # stats, real numbers only

print("--- Year counts ---")
print(df["year"].value_counts().sort_index())  # rows per year, not an average

print("--- Summary statistics: categories ---")
print(df.describe(include="object"))      # stats for text columns
```

What the above code is about

Before you can trust any dataset, you need to interrogate it — not just glance at the first few rows and assume it's fine. This block runs through five quick checks that answer five different questions.

"Are there any gaps?" isna().sum() counts how many blanks (missing values) sit in each column. A few missing measurements is normal in real-world data; a column that's mostly empty is a red flag.

"Is anything repeated by accident?" duplicated().sum() checks whether any row is an exact copy of another. Duplicates can quietly bias your results — the same penguin counted twice looks like two penguins.

"Are the categories written consistently?" unique() on species lists every distinct value that shows up. This catches the classic mess of "Adelie", "adelie", and "ADELIE " all meaning the same thing but being treated as three different categories.

"How is the data spread across groups?" value_counts() on island tells you how many penguins came from each island, so you know if one group is barely represented.

"What do the numbers actually look like?" This is the part that needed fixing. Calling describe() on the whole dataset blindly averages every numeric-looking column, including year — but "the average year was 2008.06" isn't a useful fact. So instead: measurements get describe()'d on their own, year is counted with value_counts() since it's really a label wearing a number's disguise, and describe(include="object") brings the text columns back into view, since the default describe() silently skips them.

The bigger lesson: a function running without an error doesn't mean it's telling you something meaningful. describe() will happily average a column that shouldn't be averaged — it has no idea year is a label in disguise. Reading output critically is a bigger part of data inspection than running the code in the first place.

Step 6 — Figure out an assumption

```
df[df["sex"].isna()]   # rows where sex was never recorded
```

This isn't just a missing-data check you've already done in Step 5 — it's about asking why those specific rows are missing, using what Step 2 told you about how the data was collected.

Task: write one sentence stating your assumption — for example, "we assume sex is missing because it wasn't safely determinable in the field that day, not because it doesn't apply to that penguin." That's a genuine assumption: the data can't confirm it, but it's the most reasonable read given the collection method.

Step 7 — Cluster without the label

```
from sklearn.preprocessing import StandardScaler  # rescales features to comparable ranges
from sklearn.cluster import KMeans                 # groups similar rows together

features = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]
clean = df.dropna(subset=features)                 # drop rows missing any of these

scaler = StandardScaler()
X_scaled = scaler.fit_transform(clean[features])   # rescale so no feature dominates

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clean["cluster"] = kmeans.fit_predict(X_scaled)     # assign each row to a cluster

pd.crosstab(clean["cluster"], clean["species"])     # compare clusters to the real species
```

This groups penguins purely by their measurements, without the model ever being told the species. StandardScaler puts all four measurements on the same scale first — otherwise body_mass_g (values in the thousands) would dominate bill_depth_mm (values around 15-20) just because its numbers are bigger, not because it's more important. KMeans then sorts the rows into 3 groups based on how similar their scaled measurements are.

Task: look at the crosstab — do the 3 clusters line up closely with the 3 real species, even though the model never saw the species column? That would suggest species really is reflected in body shape alone.

Step 8 — Predict & check accuracy

```
from sklearn.model_selection import train_test_split  # splits data into train/test sets
from sklearn.tree import DecisionTreeClassifier         # the model we'll train
from sklearn.metrics import accuracy_score               # scores how many predictions were
correct

X = clean[features]        # inputs (the measurements)
y = clean["species"]       # the label we're trying to predict

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
```

```
)                           # 70% train, 30% test, keep species proportions balanced

model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)   # learn patterns from the training data

predictions = model.predict(X_test)   # guess species on data it hasn't seen
print("Accuracy:", round(accuracy_score(y_test, predictions) * 100, 2), "%")
```

This is different from Step 7 in one key way: the model is given the species label during training, so it can learn the relationship between measurements and species directly, rather than just finding groups on its own. train_test_split holds back 30% of the data that the model never trains on, so accuracy_score is measuring performance on penguins it hasn't memorised the answer for.

Task: before running the last cell, guess whether accuracy will be high or low — penguin species tend to have quite distinct measurements, so a strong result here would make sense. Then run it and see if your guess was right.